

CPSC 250

Dictionaries

Edward Brash

1 Introduction

Dictionaries are one of the most powerful and useful data structures in Python.

Many computer scientists use dictionaries constantly because they provide an efficient way to map one piece of information to another.

Students often struggle with dictionaries initially, but they are extremely useful once understood properly.

2 When Should You Use a Dictionary?

A dictionary is useful whenever there exists a natural one-to-one mapping between two collections of information.

For example:

- names $\rightarrow$ ages
- student IDs $\rightarrow$ GPAs
- usernames $\rightarrow$ passwords
- countries $\rightarrow$ capitals

If there is a relationship between two kinds of data, a dictionary may be an appropriate structure.

3 Example Using Two Lists

Suppose we initially store information in two separate lists:

```
names = ['Bob', 'Alice', 'Jane', 'Fred']
ages = [18, 31, 16, 47]
```

The problem is that the relationship between the lists must be maintained manually.

For example:

- names[0] corresponds to ages[0]
- names[1] corresponds to ages[1]

This becomes cumbersome and error-prone.

4 Replacing with a Dictionary

Instead, we can directly store the relationship:

```
people = {  
    'Bob': 18,  
    'Alice': 31,  
    'Jane': 16,  
    'Fred': 47  
}
```

This preserves the correlation between the data naturally.

5 Accessing Dictionary Values

Dictionary values are accessed using keys:

```
print(people['Bob'])
```

Output:

```
18
```

6 Important Terminology

In a dictionary:

```
'Bob' : 18
```

- 'Bob' is called the **key**
- 18 is called the **value**

Dictionaries store **key/value pairs**.

7 Keys and Values Can Have Different Types

Keys and values do not need to have the same type.

Example:

```
friends = {  
    'Bob': 'Alice',  
    'Jane': 'Heather',  
    'Fred': 'John'  
}
```

Another example:

```
finish_order = {  
    1: 'Bob',  
    2: 'Fred',  
    3: 'Bill'  
}
```

```
print(finish_order[2])
```

Output:

```
Fred
```

Important:

The value inside square brackets is a **key**, not an index.

8 Useful Dictionary Methods

- `my_dict.keys()` — return all keys
- `my_dict.values()` — return all values
- `my_dict.items()` — return all key/value pairs
- `my_dict.get(key)` — safely retrieve a value
- `my_dict.update(other_dict)` — merge dictionaries
- `my_dict.pop(key)` — remove a key/value pair

9 Mutability

Dictionaries are mutable structures.

This means:

- values can be changed,
- new entries can be added,
- entries can be removed.

Example:

```
people['Bob'] = 19  
people['Mary'] = 27
```

10 Summary of Python Data Structures

Type	Description	Mutable?
int	Numeric integer	Yes*
float	Floating point number	Yes*
string	Sequence of characters	No
list	Ordered sequence	Yes
tuple	Ordered immutable sequence	No
set	Unordered unique collection	Yes
dict	Key/value mapping	Yes

*Python integers and floats are technically objects. Operations create new objects rather than modifying existing ones in place.

11 Key Takeaways

- Dictionaries store mappings between keys and values.
- They are ideal for representing related information.
- Keys are used for lookup instead of numeric indexing.
- Dictionaries are mutable and highly flexible.
- Dictionaries are among the most useful data structures in Python.